1. Software Requirements Specification

DICTIONARY_STDLIB

Document Control	
Author(s)	Michael Gardner
Version	0.1.0
Status	Draft
Status Date	2026-04-07
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2026 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	application
Library Role	N/A
Application Role	service
Assurance	non-spark
Execution	concurrent
Parallelism	bounded
Platform	server
Execution Environment	linux
Processor Architecture	amd64
Deployment	containerized

Table of Contents

1. Software Requirements Specification 1
2. Introduction 1
   2.1. Purpose 1
   2.2. Scope 1
   2.3. Definitions and Acronyms 1
   2.4. References 1
3. Overall Description 1
   3.1. Product Perspective 1
   3.2. Product Features 2
   3.3. User Classes 2
   3.4. Operating Environment 2
   3.5. Constraints 3
4. Interface Requirements 3
   4.1. User Interfaces 3
   4.2. Software Interfaces 3
      4.2.1. REST API 3
      4.2.2. Request/Response Formats 3
      4.2.3. Design Decisions 3
   4.3. Communications Interfaces 3
   4.4. Hardware Interfaces 4
5. Functional Requirements 4
   5.1. Key and Value Validation 4
   5.2. Dictionary Store 4
   5.3. HTTP Processing 4
   5.4. Routing and Dispatch 5
   5.5. Server and Concurrency 5
   5.6. Health Check 5
6. Quality and Cross-Cutting Requirements 5
   6.1. Performance 5
   6.2. Reliability 6
   6.3. Portability 6
   6.4. Maintainability 6
   6.5. Testability 6
   6.6. Containerization 6
7. Design and Implementation Constraints 6
   7.1. System Requirements 6
   7.2. Architectural Constraints 7
8. Verification and Traceability 7
   8.1. Verification Methods 7
   8.2. Traceability Matrix 7
9. Appendices 8
   9.1. Appendix A: Package Responsibilities 8
   9.2. Change History 8

2. Introduction

2.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **Dictionary_Stdlib**, an educational REST microservice built exclusively with Ada 2022 standard library packages.

2.2. Scope

Dictionary_Stdlib is intended to provide:

- A working RESTful HTTP/1.1 key-value dictionary service.
- A teaching tool demonstrating Ada 2022 features: generics, contracts, protected objects, tasking, and bounded types.
- HTTP protocol handling built from raw TCP sockets (GNAT.Sockets), showing what frameworks abstract away.
- Hand-rolled JSON parsing and serialization for simple payloads.
- A foundation for two follow-on projects: a crate-based version and a full hybrid enterprise version.

This project intentionally omits production-level features (TLS, persistent storage, graceful shutdown, structured logging) to keep the focus on core language and protocol concepts.

2.3. Definitions and Acronyms

Term	Definition
Bounded Type	A type with a fixed maximum size, requiring no heap allocation.
Contract	Pre/Post conditions and type invariants enforced at compile time or runtime.
CRUD	Create, Read, Update, Delete – the four basic data operations.
Generic	An Ada parameterized package or subprogram instantiated at compile time.
GNAT.Sockets	GNAT runtime library for TCP/UDP socket programming.
Ordered_Maps	Ada standard container maintaining keys in sorted order.
Protected Object	Ada concurrency primitive providing built-in reader/writer locking.
REST	Representational State Transfer – an architectural style for HTTP APIs.
Task	Ada's built-in unit of concurrent execution (similar to a thread).

2.4. References

- Ada 2022 Language Reference Manual (ISO/IEC 8652:2023).
- GNAT Reference Manual (GNAT.Sockets specification).
- RFC 7231 – HTTP/1.1 Semantics and Content.

3. Overall Description

3.1. Product Perspective

Dictionary_Stdlib is a standalone HTTP microservice that exposes a key-value dictionary through a RESTful API. It is the first of three planned implementations:

1. **Dictionary_Stdlib** (this project) – standard library only, educational.
2. **Dictionary_Crates** – same API using Ada Web Server and JSON crates.
3. **Dictionary_Hybrid** – full DDD/Clean/Hexagonal enterprise architecture.

The service has no external dependencies beyond the Ada 2022 standard library and the GNAT runtime. It is not layered according to the hybrid architecture; it uses a flat package structure organized by concern.

Dictionary_Main (entry point) Starts the server on the configured port.
|
Dictionary.Server (TCP listener + tasking) Accepts connections, spawns a task per client.
|
Dictionary.Router (static dispatch) Matches method + path, delegates to handlers.
|
Dictionary.HTTP / JSON / Store / Validation / Types Parsing, serialization, storage, validation, bounded types.

3.2. Product Features

1. **RESTful CRUD API**: POST, GET, PUT, DELETE on dictionary entries, plus a health check endpoint.
2. **Case-Insensitive Keys**: Keys are normalized to lowercase at construction time.
3. **Bounded In-Memory Store**: Maximum 100 entries, thread-safe via Ada protected object.
4. **HTTP/1.1 from Scratch**: Hand-parsed request line, headers, and body over raw TCP.
5. **Hand-Rolled JSON**: Minimal parse/serialize for the service's two payload shapes.
6. **Concurrent Request Handling**: Task-per-connection using Ada native tasking.
7. **Generic Bounded Text**: Reusable generic parameterized by max length and character predicate.
8. **Contract-Based Validation**: Pre/Post conditions on all public validation functions.
9. **Docker Production Image**: Multi-stage build producing a minimal nonroot container.

3.3. User Classes

User Class	Description
API Consumers	Developers or tools sending HTTP requests to the service (curl, Postman, application code).
Learners	Engineers studying the source code to understand Ada 2022, HTTP, and microservice patterns.
Maintainers	Developers extending or porting the service to the crate-based or hybrid versions.

3.4. Operating Environment

Requirement	Specification
Platform	Linux (amd64)
Runtime	Docker container or bare metal

Requirement	Specification
Ada Version	Ada 2022 (GNAT FSF 15.x via Alire)
Build System	GPRbuild via Alire
Dependencies	None beyond the Ada standard library and GNAT runtime
Port	TCP 8080 (default)

3.5. Constraints

Constraint	Rationale
Bounded store (100 entries)	Teaches resource-limit awareness with bounded data structures.
HTTP only (no TLS)	TLS requires C bindings or third-party libraries, outside scope.
In-memory storage only	Simplicity; data is lost on restart. Production would use a database.
No graceful shutdown	Ctrl+C terminates the process. Production would drain connections.
Standard library only	Educational goal: show what can be built without third-party crates.

4. Interface Requirements

4.1. User Interfaces

The service has no graphical interface. All interaction is through HTTP requests and JSON responses on port 8080.

4.2. Software Interfaces

4.2.1. REST API

Method	Path	Purpose	Status Codes
DELETE	/entries/{key}	Remove an entry	204, 404
GET	/entries	List all (sorted by key)	200
GET	/entries/{key}	Retrieve one entry	200, 404
GET	/health	Health check	200
POST	/entries	Create an entry	201, 400, 409
PUT	/entries/{key}	Update value (strict)	200, 400, 404

4.2.2. Request/Response Formats

- **Create/Update body**: `{"key":"...","value":"..."}`
- **Single entry response**: `{"key":"...","value":"..."}`
- **List response**: `[{"key":"...","value":"..."},...]`
- **Error response**: `{"error":"..."}`
- **Health response**: `{"status":"healthy"}`

4.2.3. Design Decisions

- **No PATCH**: With a single mutable field (value), PATCH and PUT are identical. PUT suffices.
- **PUT is strict update**: Returns 404 if the key does not exist. POST is the only way to create.
- **Connection: close**: Every response closes the connection. No keep-alive.

4.3. Communications Interfaces

TCP socket on port 8080. HTTP/1.1 protocol. No TLS.

4.4. Hardware Interfaces

None.

5. Functional Requirements

5.1. Key and Value Validation

Priority: High
Description: Input validation with explicit contracts.

ID	Requirement
FR-01.1	Keys shall consist of alphanumeric characters and hyphens only (`[a-zA-Z0-9-]+`).
FR-01.2	Keys shall be between 1 and 50 characters in length.
FR-01.3	Keys shall be case-insensitive, normalized to lowercase on storage.
FR-01.4	Values shall consist of printable ASCII characters (space through tilde).
FR-01.5	Values shall be between 1 and 200 characters in length.
FR-01.6	Validation functions shall use Pre/Post contracts to document constraints.

Acceptance Criteria:

- Invalid keys or values produce a 400 Bad Request with a descriptive error message.
- Duplicate keys (case-insensitive) produce a 409 Conflict.

5.2. Dictionary Store

Priority: High
Description: Thread-safe bounded key-value store.

ID	Requirement
FR-02.1	The store shall hold a maximum of 100 entries.
FR-02.2	The store shall use a protected object for thread safety.
FR-02.3	Read operations (Get, Get_All, Contains, Count) shall permit concurrent access.
FR-02.4	Write operations (Create, Update, Delete) shall require exclusive access.
FR-02.5	Get_All shall return entries sorted by key in ascending order.
FR-02.6	Keys shall be unique; duplicate insertion shall return Already_Exists status.

Acceptance Criteria:

- Concurrent GET requests do not block each other.
- A POST followed by a GET returns the created entry.
- Exceeding 100 entries returns a 409 Conflict with “dictionary is full” message.

5.3. HTTP Processing

Priority: High
Description: HTTP/1.1 parsing and response formatting from raw TCP.

ID	Requirement
FR-03.1	The service shall parse HTTP/1.1 request lines (method, path, version). Note: this is a deliberately simplified HTTP/1.1 subset; the version token is not validated.
FR-03.2	The service shall parse the Content-Length header to determine body size.

ID	Requirement
FR-03.3	The service shall support GET, POST, PUT, and DELETE methods.
FR-03.4	Unknown methods shall produce a Method_Unknown parse result.
FR-03.5	Responses shall include Content-Type, Content-Length, and Connection headers.
FR-03.6	All responses shall use `Connection: close`.

5.4. Routing and Dispatch

Priority: High
Description: Static dispatch routing based on method and path.

ID	Requirement
FR-04.1	The router shall match requests to handlers using method + path pattern.
FR-04.2	Unrecognized paths shall return 404 Not Found.
FR-04.3	Wrong methods on recognized paths shall return 405 Method Not Allowed.
FR-04.4	PUT with a body key that does not match the URL key shall return 400 Bad Request.

5.5. Server and Concurrency

Priority: High
Description: TCP listener with task-per-connection concurrency.

ID	Requirement
FR-05.1	The server shall listen on a configurable TCP port (default 8080).
FR-05.2	Each accepted connection shall be handled by a dedicated Ada task.
FR-05.3	A task shall read the request, dispatch through the router, send the response, and close the socket.
FR-05.4	Exceptions in a connection task shall not crash the server.
FR-05.5	The server shall set SO_REUSEADDR to allow restart without address-in-use errors.

5.6. Health Check

Priority: Medium
Description: Operational health endpoint.

ID	Requirement
FR-06.1	GET /health shall return 200 with `{"status":"healthy"}`.
FR-06.2	The health endpoint shall support only the GET method.

6. Quality and Cross-Cutting Requirements

6.1. Performance

ID	Requirement
NFR-01.1	The service shall handle concurrent requests without serializing reads.
NFR-01.2	Static dispatch (no tagged types) shall be used for routing to avoid dynamic dispatch overhead.

6.2. Reliability

ID	Requirement
NFR-02.1	A malformed request shall produce a 400 response, not a server crash.
NFR-02.2	An exception in one connection task shall not affect other connections.

6.3. Portability

ID	Requirement
NFR-03.1	The service shall use only Ada standard library and GNAT runtime packages.
NFR-03.2	The Docker image shall run on any amd64 Linux host.

6.4. Maintainability

ID	Requirement
NFR-04.1	Each package shall have a single, well-defined responsibility.
NFR-04.2	Package dependencies shall flow downward with no cycles.
NFR-04.3	Source code shall include explanatory comments targeting experienced engineers from other languages.

6.5. Testability

ID	Requirement
NFR-05.1	Unit tests shall cover validation, bounded text, JSON, HTTP parsing, and store operations.
NFR-05.2	Integration tests shall exercise the router pipeline (request to response).
NFR-05.3	End-to-end tests shall exercise all endpoints via HTTP against a running server.

6.6. Containerization

ID	Requirement
NFR-06.1	The Docker image shall use a multi-stage build (build in dev container, run in minimal image).
NFR-06.2	The runtime container shall run as a nonroot user.
NFR-06.3	The runtime container shall support a read-only root filesystem.
NFR-06.4	The image shall include a health check compatible with Docker and Kubernetes.

7. Design and Implementation Constraints

7.1. System Requirements

Item	Requirement
Hardware	Any amd64 system with Docker support
Operating System	Linux (build and runtime)
Compiler	GNAT FSF 15.x via Alire
Build Tools	GPRbuild 25.x, Alire 2.1.x, Make
Container	Docker (build and deployment)

7.2. Architectural Constraints

This project does not use the hybrid DDD/Clean/Hexagonal architecture. It uses a flat package structure organized by concern (types, validation, store, JSON, HTTP, router, server). The hybrid architecture will be used in Dictionary_Hybrid.

Dependencies flow downward only:

```
dictionary_main -> server -> router -> http, json, store, validation -> types ->
bounded_text
```

8. Verification and Traceability

8.1. Verification Methods

Method	Description
Unit Tests	99 tests covering validation, bounded text, JSON, HTTP, and store.
Integration Tests	26 tests exercising the router pipeline end-to-end.
E2E Tests	28 tests via curl against a running server.
Docker Tests	`make docker-test` builds and exercises the production image.
Build Verification	Clean build with `-gnatwa` (all warnings) and strict style checks.

8.2. Traceability Matrix

Requirement ID	Verification	Evidence
FR-01.1–FR-01.6	Unit Tests	test_validation.adb, test_bounded_text.adb
FR-02.1–FR-02.6	Unit Tests	test_store.adb
FR-03.1–FR-03.6	Unit Tests	test_http.adb
FR-04.1–FR-04.4	Integration Tests	test_router.adb
FR-05.1–FR-05.5	E2E Tests	e2e_test.sh
FR-06.1–FR-06.2	E2E Tests	e2e_test.sh
NFR-06.1–NFR-06.4	Docker Tests	make docker-test

9. Appendices

9.1. Appendix A: Package Responsibilities

Package	**Responsibility**
Dictionary	Root namespace (Pure, empty).
Dictionary.Bounded_Text	Generic validated bounded text with normalization.
Dictionary.HTTP	HTTP/1.1 request parsing and response formatting.
Dictionary.JSON	Hand-rolled JSON parse and serialize.
Dictionary.Router	Static dispatch route matching and handlers.
Dictionary.Server	TCP listener and task-per-connection.
Dictionary.Store	Thread-safe CRUD store (protected + Ordered_Maps).
Dictionary.Types	Key/Value instantiations, Entry_Record, Store_Status.
Dictionary.Validation	Character and string predicates with contracts.

9.2. Change History

Version	Date	Author	Changes
0.1.0	2026-04-07	Michael Gardner	Initial SRS for the dictionary_stdlib educational microservice.